



Deploy Backend with Kubernetes



dcowart87@gmail.com

```
[ec2-user@ip-172-31-92-116 manifests]$ kubectl apply -f flask-deployment.yaml
kubectl apply -f flask-service.yaml
deployment.apps/nextwork-flask-backend created
service/nextwork-flask-backend created
```



Introducing Today's Project!

In this project, I will prepare the backend of an application for deployment on Kubernetes because containerized applications require a properly configured deployment environment to run reliably at scale. To accomplish this, I will install and configure **kubectl**, deploy the backend application to an Amazon EKS cluster, and monitor the deployment process to verify that Kubernetes is successfully managing the application's Pods and resources.

Tools and concepts

I used **Kubernetes**, **Amazon ECR**, **Amazon EKS**, **kubectl**, and **Docker** to deploy and manage a containerized backend application in a Kubernetes cluster. Docker was used to package the application into a container image, Amazon ECR was used to store the image, Amazon EKS provided the Kubernetes environment, and kubectl was used to deploy and manage resources within the cluster. Key concepts include using **manifests** to define Kubernetes resources, **Deployments** to manage application Pods and replicas, **Services** to expose the application and route traffic, and **Pods** as the smallest deployable units in Kubernetes. This project also reinforced how container registries, Kubernetes networking, and declarative configuration work together to support scalable and reliable application deployments.

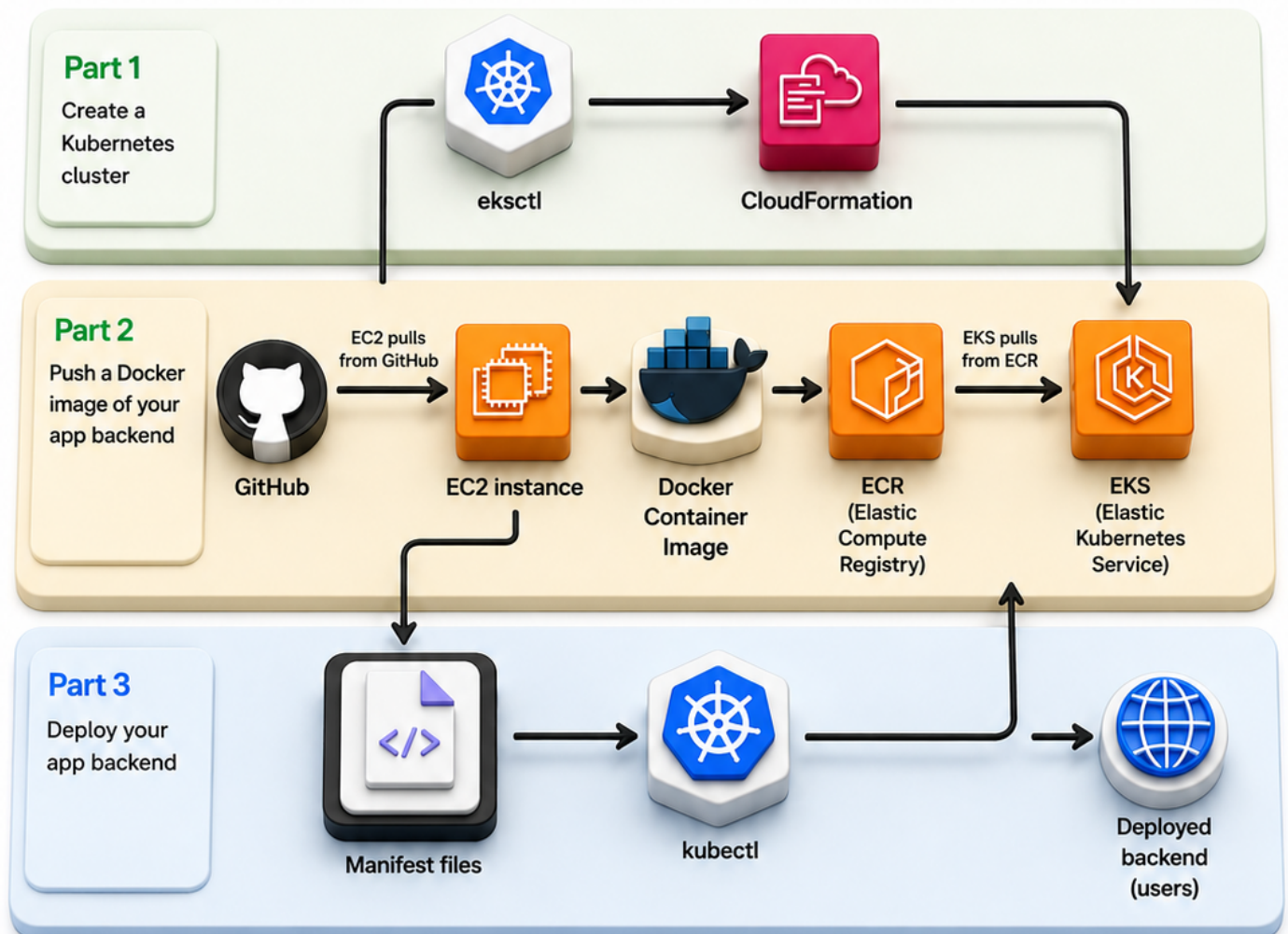
Project reflection

This project took me approximately 90 minutes to complete. After working through Parts 1–3 of the project series, I found the concepts and workflow much more familiar, so there were no major challenges during this phase. My favorite part was learning how Kubernetes Deployments, Services, and Pods work together to deploy and manage containerized applications. Seeing the application successfully deployed in Amazon EKS and verifying its status through the EKS console helped reinforce my understanding of how Kubernetes manages application availability, networking, and scaling.



Project Overview

This diagram illustrates the high-level workflow for building, deploying, and exposing a containerized backend application on Amazon EKS using Kubernetes manifests.



Part 1 | Use `eksctl` to create an Amazon EKS cluster, which provisions the necessary AWS resources using CloudFormation.

Part 2 | Source code is stored on GitHub. The EC2 instance pulls the code, builds a Docker image, pushes it to Amazon ECR, and EKS pulls the image for deployment.

Part 3 | Kubernetes manifest files (Deployment and Service) are applied using `kubectl` to deploy the application to the EKS cluster and expose it to users through a stable endpoint.



Project Set Up

Kubernetes cluster

To set up today's project, I launched a Kubernetes cluster. The cluster's role in this deployment is to provide the environment where containerized applications can be deployed, managed, scaled, and monitored. To create the cluster, I launched an EC2 instance, connected to it using EC2 Instance Connect, and installed `eksctl` through the AWS CLI. I then attached an IAM role to the instance to provide the permissions required to interact with AWS services and provision Amazon EKS resources. With these prerequisites in place, I used `eksctl` to create and deploy a Kubernetes cluster on Amazon EKS.

Backend code

To retrieve the backend application, I connected to my EC2 instance, installed Git, and cloned the application's GitHub repository. The backend is the server-side component of an application that processes requests, communicates with external services, and manages data behind the scenes. Cloning the repository provided the source code needed to build a Docker image and prepare the application for deployment in Kubernetes.

Container image

Once I cloned the backend code, I built a container image because Kubernetes deploys applications using container images rather than source code directly. A container image packages the application's code, dependencies, libraries, and configuration into a single portable artifact that can be deployed consistently across different environments. Without an image, it would be difficult for Kubernetes to reliably create identical instances of the application, scale workloads, and maintain consistent behavior across development, testing, and production environments. By building a container image, I created a standardized package that Kubernetes can use to launch and manage the application's containers.



I also pushed the container image to a container registry, which is a centralized repository used to store, manage, and distribute container images. In this project, I used Amazon ECR to securely store the backend application's container image so that Kubernetes could access it when deploying the application. ECR facilitates scaling for my deployment because Kubernetes can quickly pull the stored image whenever it needs to create additional Pods. This ensures that every new container is identical and consistently configured, allowing the application to scale efficiently while avoiding the common "it works on my machine" problem.



Manifest files

A Kubernetes manifest is a configuration file that tells Kubernetes how to deploy and manage an application. It defines details such as the container image to run, the number of replicas to create, and the resources the application requires. Manifests are essential because they allow Kubernetes to automate application deployment and management. Without them, applications would need to be configured manually each time they are deployed, making the process more time-consuming, difficult to reproduce, and prone to errors.

A Deployment manifest manages how Kubernetes deploys, updates, scales, and maintains an application within a cluster. It defines important settings such as the container image to use, the number of replicas to run, labels, and resource requirements. Kubernetes uses this information to ensure that the application remains in its desired state and automatically replaces Pods if they fail. The container image URL in my Deployment manifest tells Kubernetes which container image to pull and run when creating Pods. This enables Kubernetes to deploy identical instances of the application and consistently recreate them whenever scaling or recovery is needed.

A Service resource exposes an application running in a Kubernetes cluster by providing a stable network endpoint that routes traffic to the appropriate Pods. It acts as a traffic controller, ensuring that requests are directed to the correct Pods even if those Pods are restarted, replaced, or scaled. My Service manifest sets up a Kubernetes Service that identifies the backend Pods using label selectors and forwards incoming traffic to them on the specified port. This allows users and other applications to reliably access the backend without needing to know the changing IP addresses of individual Pods.



```
GNU nano 8.3 flask-deployment.yaml
--
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nextwork-flask-backend
  namespace: default
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nextwork-flask-backend
  template:
    metadata:
      labels:
        app: nextwork-flask-backend
    spec:
      containers:
        - name: nextwork-flask-backend
          image: 783149339342.dkr.ecr.us-east-1.amazonaws.com/nextwork-flask-backend:latest
          ports:
            - containerPort: 8080
```




Backend Deployment!

To deploy my backend application, I completed several steps from the AWS CLI. First, I installed **kubectl**, the command-line tool used to interact with Kubernetes clusters. Next, I configured the necessary access permissions to give my EC2 user access to the Amazon EKS cluster. This allowed me to use **kubectl** to interact with Kubernetes resources and deploy the application. Finally, I applied my Kubernetes manifests to create the **Deployment** and **Service** resources, enabling Kubernetes to deploy the backend application and provide network access to it.

kubectl

kubectl is the command-line tool used to interact with Kubernetes resources such as Deployments, Services, Pods, and other objects running within a cluster. I need this tool to apply my manifests, deploy the application, and manage resources after the cluster has been created. I can't use **eksctl** for the job because its primary purpose is to create, configure, and manage Amazon EKS clusters. While **eksctl** is excellent for provisioning cluster infrastructure, **kubectl** is the tool designed for deploying applications and managing resources within the Kubernetes cluster itself.



```
[ec2-user@ip-172-31-92-116 manifests]$ kubectl apply -f flask-deployment.yaml
kubectl apply -f flask-service.yaml
deployment.apps/nextwork-flask-backend created
service/nextwork-flask-backend created
```



Verifying Deployment

As an extension of this project, I used the Amazon EKS console to view my cluster and verify that the backend application had been deployed successfully. To do this, I first needed to configure access permissions because AWS IAM permissions do not automatically provide access to Kubernetes resources. Kubernetes uses its own access control system to manage permissions within a cluster. To give myself access to the cluster, I returned to my EC2 instance and ran the necessary AWS CLI commands to map my IAM identity to the appropriate Kubernetes permissions.

Once I gained access into my cluster's nodes, I discovered pods running inside each node. Pods are the smallest deployable units in Kubernetes and serve as the environment where one or more containers run together as a single unit. Kubernetes deploys and manages Pods rather than individual containers, making them a fundamental building block of the platform. Containers in a Pod share networking and storage resources, allowing them to communicate and exchange data efficiently. This shared environment helps related containers work together seamlessly while Kubernetes manages the Pod as a single entity.

The EKS console shows you the events for each Pod, where I could see the complete lifecycle of the backend container being deployed. The Events tab showed that Kubernetes successfully scheduled the Pod to a worker node, pulled the container image from Amazon ECR, created the container, and started the application successfully. This validated that the Deployment was working as expected and that the Pod was able to retrieve the container image, launch the container, and run the backend application without errors. It also confirmed that communication between Amazon EKS and Amazon ECR was functioning correctly.



Events (5)

Type ▾	Reason ▾	Event time ▾	From ▾	Message ▾
✔ Normal	Started	an hour ago	kubelet	Started container nextwork-flask-backend
✔ Normal	Pulled	an hour ago	kubelet	Successfully pulled image "783149339342.dkr.ecr.us-east-1.amazonaws.com/nextwork-flask-backend:latest" in 2.76s (2.76s including waiting). Image size: 40004077 bytes.
✔ Normal	Created	an hour ago	kubelet	Created container: nextwork-flask-backend
✔ Normal	Pulling	an hour ago	kubelet	Pulling image "783149339342.dkr.ecr.us-east-1.amazonaws.com/nextwork-flask-backend:latest"
✔ Normal	Scheduled	an hour ago	default-scheduler	Successfully assigned default/nextwork-flask-backend-cc84c7cc8-27dbr to ip-192-168-11-82.ec2.internal



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

